

EXAMEN DE FIN D'ÉTUDES SECONDAIRES CLASSIQUES

Sessions 2023 – QUESTIONNAIRE ÉCRIT

Date :	08.06.23	Durée :	08:15 - 11:15	Numéro candidat :	
Discipline :	Science de la programmation		Section(s) :	CI	

Partie modélisation

10 points

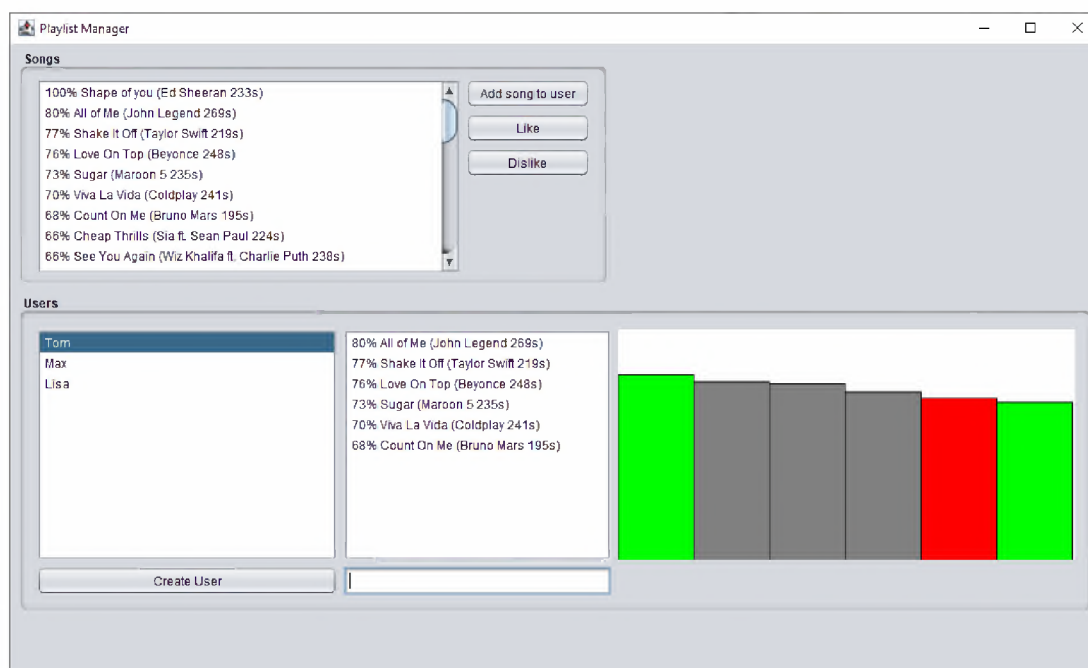
- La partie modélisation doit être rendue avant que la partie implémentation ne puisse être débutée. Cette partie est à réaliser sur papier.
- La partie modélisation est à rendre après 35 minutes au plus tard, mais vous êtes libre de la rendre plus tôt.

Un propriétaire de plateforme musicale veut offrir un programme qui permet de créer, de gérer et de personnaliser une liste de lecture de musique pour plusieurs utilisateurs.

Principe de l'application

L'application contient plusieurs chansons. Une chanson a un titre, le nom de l'artiste, la durée et plusieurs scores. Un score est soit un 0, ce qui représente un « je n'aime pas » soit un 100 ce qui représente un « j'aime ». Ces chansons sont stockées dans un fichier texte, chaque ligne de ce fichier a le format suivant : <titre>,<artiste>,<durée>,<score1;score2;... ;scoreN>

Cette application gère plusieurs utilisateurs qui ont des noms uniques et chaque utilisateur possède une liste de chansons qu'il peut gérer. La liste de chansons ne peut pas contenir la même chanson plusieurs fois.



La plateforme souhaite une application avec les fonctionnalités suivantes :

1. Au lancement, l'application lit toutes les chansons depuis un fichier texte. Les chansons sont affichées sous forme textuelle dans une liste. Les chansons sont triées selon leurs scores.
2. Le bouton « **Add song to playlist** » permet d'ajouter une chanson à la collection de l'utilisateur sélectionné. Une collection de chansons d'un utilisateur ne peut que contenir une chanson qu'une seule fois.
3. Le bouton « **Create User** » permet de créer un utilisateur avec le nom rentré dans le champ de texte.
4. Les utilisateurs créés sont affichés sous forme textuelle dans une liste et peuvent être sélectionnés pour ajouter des chansons à leurs listes de chansons et pour donner un « like » ou un « dislike » à une chanson.
5. En cliquant sur le nom d'un utilisateur dans la liste d'utilisateurs, les chansons de celui-ci sont affichées sous forme textuelle dans une liste et représenté dans un diagramme. Ce diagramme sert à comparer visuellement les scores moyens des chansons.

Travail à réaliser

Réaliser un modèle de classe du modèle de l'application sur papier en respectant les conventions UML usuelles.

1. Limitez-vous uniquement aux attributs des classes.
2. Rajoutez la/les méthode(s) nécessaire(s) pour trier les chansons en utilisant la méthode **Collections.sort()** de Java.
3. Rajoutez la/les méthode(s) nécessaire(s) pour afficher toutes les chansons, tous les utilisateurs et les chansons d'un utilisateur sélectionné sous forme textuelle dans les listes du type « JList ».
4. Rajoutez la/les méthode(s) nécessaire(s) pour lire les chansons depuis un fichier texte.
5. Rajoutez la/les méthode(s) nécessaire(s) pour pouvoir dessiner le diagramme.
6. Vous pouvez compléter votre modèle de classe par des explications supplémentaires.

Dans votre répertoire de travail (à définir par chaque lycée), vous trouverez un dossier nommé EXAMEN_CI. Renommez ce dossier en remplaçant le nom actuel par votre code de l'examen (exemple de notation : LXY_CI2_05). Tous vos fichiers devront être sauvegardés à l'intérieur de ce dossier, qui sera appelé votre dossier par la suite.

Partie implémentation

50 points

Vous allez programmer dans le suivant une application qui permet de gérer plusieurs utilisateurs. Vous trouvez dans votre dossier le projet **UserPlaylistManager**, que vous devez compléter suivant les indications suivantes :

La classe « Node »

(1.5 point)

La classe **Node** représente un nœud d'une liste chaînée permettant de stocker des scores. Implémentez la classe **Node** selon le schéma UML. Le **constructeur** initialise la valeur de l'attribut **data**. Elle possède les attributs suivants :

1.25 point

- **next** représente le nœud suivant
- **data** représente le score

La classe possède les méthodes suivantes :

- La méthode **hasNext()** retourne **true** si le nœud possède un prochain nœud.

0.25 points

La classe « Ratings »

(8 points)

La classe **Ratings** contient des scores. Un score peut avoir la valeur 100, ce qui représente un « j'aime », ou 0, ce qui représente un « je n'aime pas ». Elle possède l'attribut suivant :

0.5 point

- **root** représente le premier nœud de la liste chaînée

La classe possède les méthodes suivantes :

- Les méthodes **add(...)** ajoutent de manière récursif un score à la fin de la liste. **3.5 points**
- La méthode **size()** retourne le nombre d'éléments dans la liste. **2 points**
- **calculateAverageRating()** est une méthode qui calcule et retourne la moyenne des scores de la liste. **2 points**

La classe « Song »**(6.5 points)**

La classe **Song** représente une chanson qui implémente l'interface **Comparable<Song>**. Implémentez la classe **Song** selon le schéma UML. Le **constructeur** initialise la valeur des attributs **title**, **artist**, **duration**, et **ratings**. La classe possède les attributs suivants : **0.75 point**

- **title** représente le titre de la chanson
- **artist** correspond à l'artiste de la chanson
- **duration** indique la durée en secondes
- **ratings** est un objet qui contient les scores de la chanson

La classe possède les méthodes suivantes :

- **addRating(int)** est une méthode qui ajoute un score à la fin de la liste. **0.5 point**
- La méthode **getAverageRating()** retourne la moyenne de tous les scores sous forme de pourcentage. **0.5 point**
- La méthode **compareTo(Song)** permet de faire la comparaison entre deux chansons sur base de leurs scores moyens. Si les scores moyens sont égaux, les titres sont comparés en ordre décroissant. **4 points**
- La méthode **toString()** retourne une représentation textuelle de la chanson. **0.75 point**
 - **Exemple :**
<averageRating>% <titre> (<auteur> <duration>s)

La classe « User »**(13 points)**

La classe **User** représente un utilisateur qui peut créer et manipuler une liste de musique. Le **constructeur** initialise la valeur de l'attribut **name**. Complétez la classe **User** selon le schéma UML. Elle possède les attributs, accesseurs suivants : **1.5 point**

- **name** représente le nom de l'utilisateur
- **hsUserSongs** est un ensemble qui représente la liste de chansons de l'utilisateur
- **hmRatedSongs** est une table associative, qui associe une chanson à un score, pour sauvegarder à quelles chansons l'utilisateur a déjà donné un score

La classe possède les méthodes suivantes :

- **addSong(Song)** est une méthode prend comme paramètre une chanson et l'ajoute à l'ensemble de chansons. Cette méthode lance une exception du type **InvalidUserOperation**¹ avec un message significatif, si la chanson fait déjà partie de l'ensemble. **1 point**
- La méthode **like(Song)** qui prend comme paramètre une chanson. Elle ajoute un score de **100** à la chanson. Dans l'utilisateur elle associe la chanson au score **100**. En plus elle ajoute la chanson à son ensemble de chansons. Cette méthode lance une exception du type **RatingException** avec un message significatif, si l'utilisateur a déjà donné un score à cette chanson. **2 points**

¹ La classe **InvalidUserOperation** et **RatingException** vous sont mises à disposition.

- La méthode **dislike(Song)** qui prend comme paramètre une chanson. Elle ajoute un score de **0** à la chanson. Dans l'utilisateur elle associe la chanson au score **0**. En plus elle retire la chanson de son ensemble de chansons. Cette méthode lance une exception du type **RatingException** avec un message significatif, si l'utilisateur a déjà donné un score à cette chanson. **0.5 points**
- La méthode **sortedSongsList()** cette méthode retourne l'ensemble de chansons comme liste de chanson, trié selon leurs scores moyens. **2 points**
- La méthode **draw(Graphics,int,int)** dessine un diagramme à bandes (DE : Balkendiagramm). Chaque bande du diagramme correspond à une chanson. Cette méthode dessine autant de bandes qu'il y a de chansons dans l'ensemble. Les bandes ont toutes la même largeur et occupent toute la largeur de l'aire de dessin. La hauteur et la largeur de l'aire de dessin sont passés en paramètre. La hauteur d'une bande est déterminée à l'aide du score moyen d'une chanson.

Exemple : La bande pour une chanson avec un score moyen de 70 remplit l'aire de dessin à 70% de la hauteur de l'aire de dessin.

Si l'utilisateur a donné un score de **100** à une chanson, alors le corps de la bande est dessiné en **vert**, si l'utilisateur a donné un score de **0**, alors le corps de la bande est dessiné en **rouge**. Si l'utilisateur n'a pas encore donné de score, alors le corps est dessiné en **gris**. Chaque bande a un bord noir. Les bandes sont dessinées dans l'ordre des scores moyens des chansons.

5.5 points

- La méthode **toArray()** retourne un tableau d'objets contenant les chansons de l'ensemble trié selon leurs scores moyens. **0.5 point**

La classe « *ApplicationManager* »

(9 points)

La classe **ApplicationManager** permet de gérer les chansons ainsi que les utilisateurs. Elle possède les attributs suivants :

0.5 point

- **alSongs** représente la liste qui contient toutes les chansons
- **hmUsers** est une table associative, qui associe le nom à un utilisateur

La classe possède aussi les méthodes suivantes :

- La méthode **getSong(int)** retourne la chanson à l'index passé en paramètre. **0.25 point**
- La méthode **getUser(String)** retourne l'utilisateur associé au nom passé en paramètre. **0.25 point**
- **loadSongsFromFile(String)** est une méthode qui lit les chansons depuis le fichier texte passé en paramètre. Chaque ligne correspond à une chanson. Une ligne a le format suivant :
<titre>,<artiste>,<durée>,<score1;score2;... ;scoreN>
Ensuite la liste de chansons est triée. **5 points**
- La méthode **addUser(User)** ajoute à la table associative l'utilisateur passé en paramètre. Si le nom est déjà utilisé, alors cette méthode lance une exception du type **InvalidUserOperation** avec un message significatif **1 point**
- La méthode **like(Song, User)** fait appel à la méthode **like()** de l'utilisateur passé en paramètre avec la chanson passée en paramètre. Ensuite la liste de chansons est triée. **0.5 point**
- La méthode **dislike(Song, User)** fait appel à la méthode **dislike()** de l'utilisateur passé en paramètre avec la chanson passée en paramètre. Ensuite la liste de chansons est triée.

- La méthode **getSongsArray()** retourne un tableau d'objets qui correspondent aux chansons de la liste de chansons 0.5 point
- La méthode **getUsersArray()** retourne un tableau d'objets qui correspondent aux noms des utilisateurs de la table associative. 0.5 point

La classe « DrawPanel » (0 point)

La classe **DrawPanel** est mise à votre disposition et doit être complétée. La classe dessine le diagramme à bandes des votes relatives aux chansons d'un utilisateur. Elle possède l'attribut du type **User** et le manipulateur correspondant. Cette classe vous est fournie.

La classe « MainFrame » (12 points)

L'interface graphique ainsi que la classe **MainFrame** avec du code squelette vous sont fournies. Complétez la classe avec les instructions suivantes :

Toutes les exceptions lancées doivent être affichées dans une fenêtre à l'aide du code suivant :

```
JOptionPane.showMessageDialog(this, "message", "title", JOptionPane.ERROR_MESSAGE);
```

Pour lire le nom d'un élément dans une **JList** utilisez le code suivant :

```
<JList>.getSelectedValue().toString()
```

Ceci vous est fourni dans la classe **MainFrame**.

- L'attribut **applicationManager** représente le modèle de l'application. 0.5 point
- Le constructeur charge les chansons à l'aide du **applicationManager** et actualise ensuite la liste de chansons **songsList**. 1 point
- La méthode **updateView()** actualise la liste de chansons **songsList**. Si un utilisateur est sélectionné, alors sa liste de chansons est aussi actualisée. Ensuite l'aire de dessin est repeinte. 2 points
- La méthode **checkUserAndSongSelected()** contrôle si un utilisateur et une chanson de la liste **songsList** sont sélectionnés, et retourne **true** si ceci est le cas. Le cas contraire cette méthode affiche le message : « You need to choose a user and a song first! » dans une fenêtre et retourne **false**. 1.5 points
- Le bouton « **Add song to user** » contrôle si un utilisateur et une chanson de la liste **songsList** sont sélectionnés. Ensuite la chanson est ajoutée à l'utilisateur sélectionné et l'interface est actualisée. 2 points
- Lorsque la sélection d'un utilisateur change, l'utilisateur sélectionné est passé au **drawPanel** 1 point
- Le bouton « **Like** » contrôle si un utilisateur et une chanson de la liste **songsList** sont sélectionnés. Ensuite il fait appel à la méthode **like(Song)** de **applicationManager** avec la chanson et l'utilisateur sélectionné. Ensuite l'interface est actualisée. 2 points
- Le bouton « **Dislike** » contrôle si un utilisateur et une chanson de la liste **songsList** sont sélectionnés. Ensuite il fait appel à la méthode **dislike(Song)** de **applicationManager** avec la chanson et l'utilisateur sélectionné. Ensuite l'interface est actualisée. 0.5 point
- Le bouton « **Create User** » crée un utilisateur avec le nom indiqué dans **userNameTextField**. Ensuite la liste d'utilisateurs **userList** est actualisée. 1.5 point

Schéma UML du modèle à implémenter

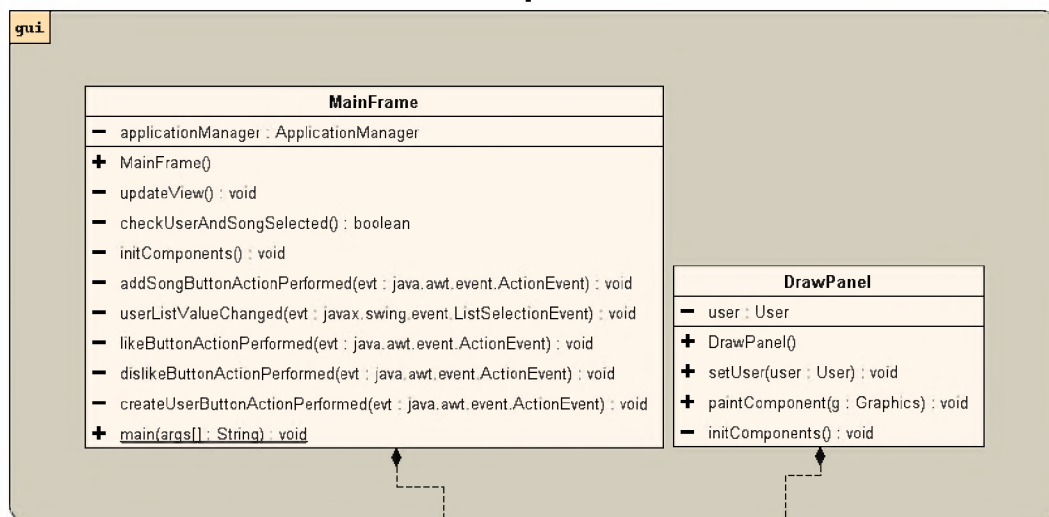


Figure 1 : Schéma UML (simplifié !) du paquet gui

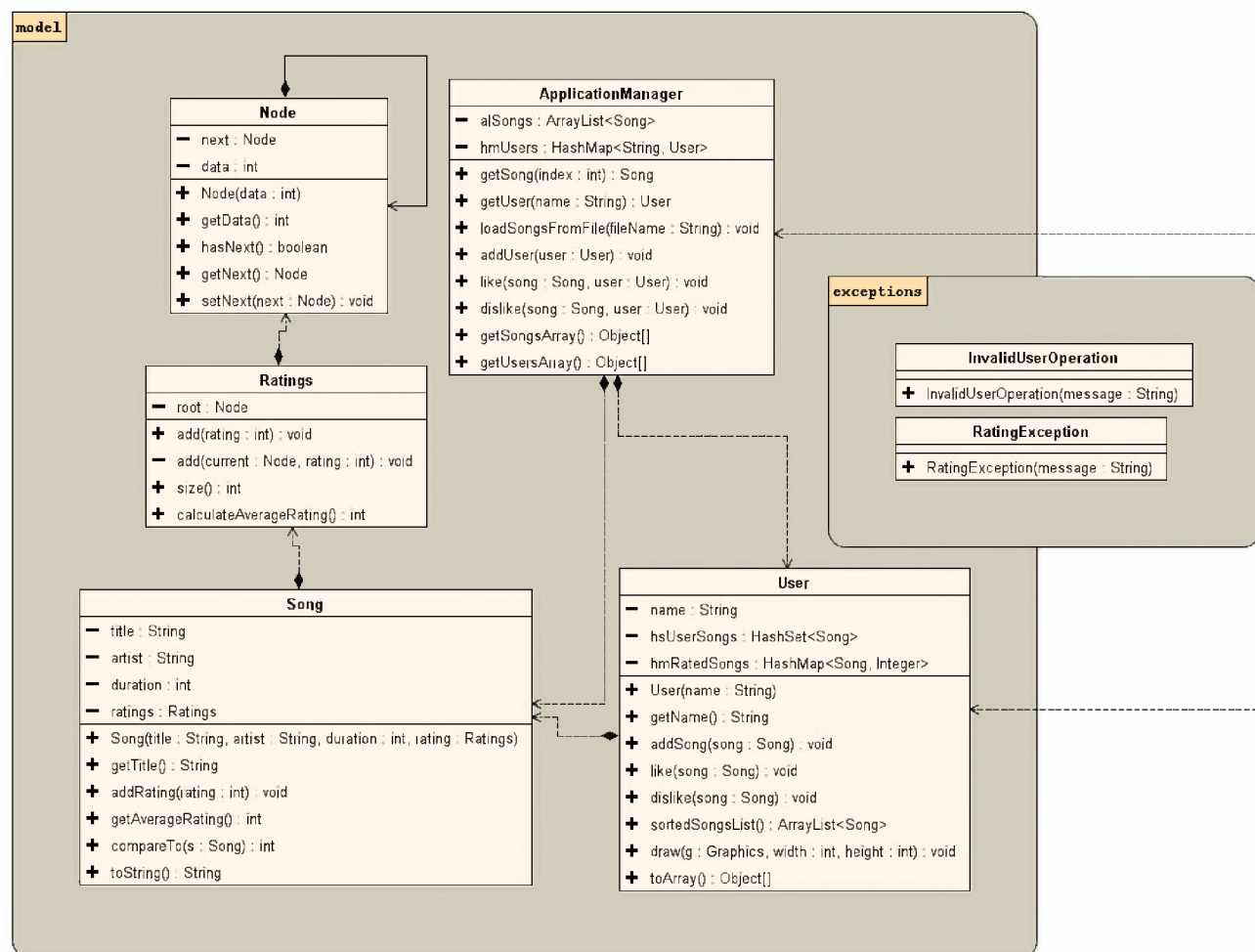


Figure 2 : Schéma UML des paquets model et exceptions. Afin de ne pas surcharger le présent schéma, les relations du type « utilise » ont été supprimées